College Event Kit Relay - Low-Level Design

Revision 4 (consolidated, implementation-ready)

Supersedes Revision 2 and Revision 3 · Oct 10, 2026 · @pragnya

Revision 4 is one consolidated specification. It keeps the design of Revision 2 and the explicit behaviour of Revision 3, and applies the review fixes listed below. Where Revision 2 and Revision 3 disagreed, the decision in this revision is final. Section 15 lists every resolved conflict.

What changed from Revision 3

#	Change	What it does
1	Title and counts corrected	Title spelling fixed. The enumeration section lists eight enumerations and says so.
2	Settings have owners	All kit settings now live on EventKit. decisionWindowSeconds lives on EventKitManager. No-show deadline is booked start plus noShowGraceMinutes.
3	Operational state complete	IN_INSPECTION -> READY is added for a passing inspection with no issues. Full transition table in section 8.
4	One error table	Revision 2 and Revision 3 errors are merged into one table (section 10). Nothing from Revision 2 is dropped.
5	Verification owner set	EventKit owns the current verification and its invalidation. InspectionReport only stores the check results and time.
6	Lost-window rule stated	A CONFLICT rejection does not change lostWindowsInRow. Only LOST_PRIORITY does.
7	One OPEN issue per item	A repeat failure adds a note to the existing OPEN issue. Example A and the workflow are updated to match.
8	Closing an issue defined	An issue closes only when its item is GOOD again, or when a replacement is recorded and set to GOOD. No other exception.
9	Late returns defined	A late return is counted in lateReturns and is not a good return. It is not a penalty. The next handover waits for the physical return, inspection and verification.
10	EXPIRED counts in reliability	An expired (no-show) reservation counts as not good: reliability uses completed plus expired reservations.
11	Bounded wait defined	A caller waiting for a request result gets RequestPendingError (retryable) instead of waiting forever.

1. Problem Statement

The system lets college clubs reserve one shared event kit for a chosen time period, and guarantees that two clubs never hold it for overlapping periods. The kit holds items such as a camera, a tripod and a microphone. The system must:

- check availability for the whole requested period and keep a 15-minute gap between bookings
- confirm or reject each reservation
- when several clubs ask for overlapping periods at the same time, decide by each club's past return history
- record when the kit is handed over and when it is returned
- verify that every item works before each handover, and inspect the items after return
- track every missing or damaged item until an organizer resolves and closes the issue
- handle no-shows, late returns and early returns so that the kit's schedule stays accurate

2. Scope and Assumptions

The design covers reservation, handover, return, inspection and issue tracking for one shared kit used by many clubs. Fees, penalties, login, notifications and a persistent database are out of scope. Storage is in memory unless an implementation adds persistence.

Assumptions taken from the requirement document:

- The kit is shared by multiple clubs.
- A reservation covers the whole kit. Items are not reserved one by one; each item's condition is tracked.
- An organizer records handover, return, inspection and issue resolution.

Where the requirement left room, these decisions apply:

Topic	Decision
Kit status flag	KitStatus is a summary only: RESERVED while at least one CONFIRMED reservation exists, otherwise AVAILABLE. It is not used to decide availability. Physical state is held separately in KitOperationalStatus.
Overlap rule	Two bookings clash when new start < existing end + bufferMinutes AND new end + bufferMinutes > existing start. bufferMinutes is 15 by default. A gap of exactly 15 minutes is allowed; a shorter gap is not.
Reservations that block	CONFIRMED and COMPLETED block their period and the buffer on each side. CANCELLED and EXPIRED do not block.
Draft reservation	A new reservation has no status until it is confirmed. A request waiting for its decision window is a BookingRequest, never a Reservation.
Cancellation	Allowed only while CONFIRMED and before handover. The actor must be the owning club or an organizer (rule 14).
Inspection timing	Items are inspected at return. Items the organizer does not mention keep their current condition.
Issue creation	Any missing, damaged or not-working item found at return, or in a failed pre-handover check, opens one OPEN issue for that item. A repeat failure for the same item adds a note to the existing issue.
Closing an issue	Two steps: record resolution notes, then close. Closing needs notes and needs the item to be GOOD again (directly, or by recording a replacement set to GOOD).
No-show	A CONFIRMED reservation with no handover becomes EXPIRED at booked start + noShowGraceMinutes (30 by default). It releases its period and counts as not good in the club's history.
Late return	A return after the booked end is recorded as late. It is counted in lateReturns and is not a good return. There is no penalty. The next handover is blocked until the kit is physically back, inspected and verified.
Early return	A return before the booked end keeps the period blocked, unless releaseUnusedTimeAfterEarlyReturn is true and the post-return inspection passes. Then the blocked period ends at the return time plus bufferMinutes.
Times	Plain date-times without time zones. Times are compared in one local clock.
Gap between bookings	A kit needs bufferMinutes (15) between bookings to return, inspect and verify it. This is a kit setting.
Verification before handover	The organizer checks every item (camera, tripod, microphone) and gives a result for each. Handover is allowed only when all are GOOD and the verification is still valid. A failed check opens or updates an issue and blocks handover until a new check passes.
Verification validity	A passing verification is valid for verificationValidityMinutes (60 by default). It is invalidated at once if any item's condition changes.
Responsibility for damage	A problem found at return is charged to the club that held the kit. A problem found in a failed pre-handover check is charged to no club.
Simultaneous requests	Requests for the same kit that arrive inside one decision window count as simultaneous. They are ranked by return history (section 9), the best is confirmed first, and the others are confirmed only if they no longer clash.
Decision window	A setting on the manager, 5 seconds by default. 0 means every request is decided at once in arrival order. A request that overlaps a booking already confirmed is rejected at once, without waiting, because a confirmed booking is never taken away.
Booking limits	A start in the past is rejected. Duration must be between minimumBookingMinutes (30) and maximumBookingDays (3 days).
Handover timing	Handover is allowed from handoverLeadMinutes (15) before the start until the booked end, and only when the kit is physically ready.

Topic	Decision
Return history	Counted per club from reservations that reached COMPLETED or EXPIRED. A completed return is good when the kit came back on or before the booked end AND the post-return inspection found every item GOOD.
Kit back before next handover	No handover while an earlier handover of the same kit has no return recorded, however late that return is.

3. Enumerations

Eight fixed sets of values keep statuses and conditions consistent across the classes. The three reservation, kit-status and issue values come from the requirement document; the others are design choices.

Enumeration	Values	Used by
KitStatus	AVAILABLE, RESERVED	EventKit.status (summary)
KitOperationalStatus	READY, OUT, IN_INSPECTION, NEEDS_REPAIR	EventKit.operationalStatus
ReservationStatus	CONFIRMED, CANCELLED, COMPLETED, EXPIRED	Reservation.status (empty until confirmed)
DisputeStatus	OPEN, RESOLVED	DisputeResolution.status
ItemCondition	GOOD, DAMAGED, MISSING, NOT_WORKING	KitItem.condition
InspectionType	PRE_HANDOVER, POST_RETURN	InspectionReport.inspectionType, DisputeResolution.origin
RequestState	QUEUED, CONFIRMED, REJECTED	BookingRequest.state
RejectReason	CONFLICT (overlaps a booking confirmed earlier), LOST_PRIORITY (lost to a higher-ranked request in the same window)	BookingRequest.rejectReason

4. Class Design

Ten classes carry the design: the eight from the requirement document plus ReturnHistory and BookingRequest. Items marked (added) are extra members the flow needs.

4.1 Club

Kind	Member	Description
Attribute	clubId : String	Unique ID of the club
Attribute	clubName : String	Name of the club
Attribute	history : ReturnHistory (added)	The club's return record, used to rank simultaneous requests
Method	requestReservation(kitId, startTime, endTime) -> BookingRequest	Asks the manager to reserve the kit. The result (a Reservation or a rejection) is ready when the window closes or the caller's wait ends
Method	viewReservations() -> List of Reservation	Lists this club's reservations

4.2 EventKit

The shared kit that clubs reserve. It owns its items, its reservations, its physical state and its current verification.

Kind	Member	Description
Attribute	kitId : String	Unique ID of the kit
Attribute	name : String	Name of the kit
Attribute	status : KitStatus	Summary: AVAILABLE or RESERVED (not used for availability decisions)
Attribute	operationalStatus : KitOperationalStatus	Physical state: READY, OUT, IN_INSPECTION or NEEDS_REPAIR

Kind	Member	Description
Attribute	items : List of KitItem	One or more items inside the kit (composition)
Attribute	reservations : List of Reservation	Every reservation made for this kit
Attribute	currentVerification : InspectionReport (added)	The latest pre-handover check; null when none is valid
Attribute	bufferMinutes : Integer	Gap between two bookings; 15 by default
Attribute	handoverLeadMinutes : Integer	How early before start a handover may be recorded; 15 by default
Attribute	noShowGraceMinutes : Integer	Time after start before an unhanded booking expires; 30 by default
Attribute	verificationValidityMinutes : Integer	How long a passing verification stays valid; 60 by default
Attribute	minimumBookingMinutes : Integer	Shortest allowed booking; 30 by default
Attribute	maximumBookingDays : Integer	Longest allowed booking in days; 3 by default
Attribute	releaseUnusedTimeAfterEarlyReturn : Boolean	Whether an early clean return releases unused time; false by default
Method	isAvailable(startTime, endTime) -> Boolean	True when no CONFIRMED or COMPLETED reservation overlaps the period or comes within bufferMinutes of it. CANCELLED and EXPIRED are ignored
Method	canBeHandedOver(reservation, now) -> Boolean	True only when the reservation is CONFIRMED, now is inside the handover window, operationalStatus is READY, no earlier handover is outstanding and a valid verification exists
Method	markReserved() / markAvailable()	Set the KitStatus summary
Method	markOut(), markInspection(), markReady(), markNeedsRepair()	Set operationalStatus. READY -> OUT on handover; OUT -> IN_INSPECTION on return; IN_INSPECTION -> READY or NEEDS_REPAIR after inspection; NEEDS_REPAIR -> READY after resolution and a passing verification
Method	invalidateVerification()	Owner of verification validity. Sets currentVerification to null. Called whenever any item condition changes
Method	releaseUnusedPeriod(returnedAt)	Shortens the blocking period of a reservation to returnedAt, only when releaseUnusedTimeAfterEarlyReturn is true and the post-return inspection passed

4.3 KitItem

Kind	Member	Description
Attribute	itemId : String	Unique ID of the item
Attribute	itemName : String	Name of the item, such as camera, tripod or microphone
Attribute	condition : ItemCondition	GOOD, DAMAGED, MISSING or NOT_WORKING
Method	updateCondition(condition)	Updates the condition. Any change calls the kit's invalidateVerification()
Method	getItemDetails() -> Map	Returns the item's ID, name and condition

4.4 Reservation

Stores a club's reservation of the kit. It starts as an unconfirmed draft with no status.

Kind	Member	Description
Attribute	reservationId : String	Unique ID of the reservation
Attribute	clubId : String	Club that made the reservation
Attribute	kitId : String	Reserved kit
Attribute	startTime, endTime : DateTime	Requested period. endTime may be shortened by releaseUnusedPeriod

Kind	Member	Description
Attribute	status : ReservationStatus	CONFIRMED, CANCELLED, COMPLETED or EXPIRED; empty until confirmed
Method	confirm()	Checks club, kit, period, booking limits and start not in the past, then sets CONFIRMED
Method	cancel(clubId, actorType)	CONFIRMED -> CANCELLED. actorType is CLUB (must match clubId) or ORGANIZER. Refused in any other state and after handover
Method	expire(now)	CONFIRMED -> EXPIRED when now >= startTime + noShowGraceMinutes and no handover exists. Refused otherwise
Method	complete()	CONFIRMED -> COMPLETED when the kit is returned
Method	getDetails() -> Map	Returns the reservation's details

4.5 HandoverRecord

Kind	Member	Description
Attribute	recordId : String	Unique ID of the record
Attribute	reservationId : String	Related reservation
Attribute	givenAt : DateTime	Time of handover
Attribute	returnedAt : DateTime	Time of physical return; empty until returned
Method	recordHandover(givenAt)	Stores the handover time. Allowed once; givenAt must be inside the handover window
Method	recordReturn(returnedAt)	Stores the actual physical return time. Needs a handover first, allowed once, never earlier than the handover

4.6 InspectionReport

Records the condition of the kit items, before a handover or after a return. It stores data only; validity is decided by EventKit (section 4.2).

Kind	Member	Description
Attribute	reportId : String	Unique ID of the report
Attribute	reservationId : String	Related reservation
Attribute	conditionNotes : String	Notes about item condition
Attribute	missingOrDamagedItems : List	Each entry holds item ID, name, condition and details
Attribute	inspectionType : InspectionType	PRE_HANDOVER or POST_RETURN
Attribute	verifiedAt : DateTime (added)	Time of a pre-handover check; used for validity
Method	inspectItems(observed conditions, notes)	Checks every kit item once, applies the observed conditions and records each item that is not GOOD
Method	verifyItems(results, notes, verifiedAt)	Pre-handover check. Needs a result for every item; records each item not GOOD; passes only when all are GOOD
Method	recordDamageOrMissing(item, details)	Adds one missing, damaged or not-working item. A GOOD item is refused. A repeat for an item already listed adds a note only

4.7 DisputeResolution

Tracks a missing-item or damage issue until an organizer resolves it and closes it. There is one OPEN issue per item at a time.

Kind	Member	Description
Attribute	disputeId : String	Unique ID of the issue

Kind	Member	Description
Attribute	reservationId : String	Related reservation
Attribute	itemId : String (added)	The item the issue is about; one OPEN issue per item
Attribute	issue : String	Description of the problem
Attribute	status : DisputeStatus	OPEN or RESOLVED
Attribute	resolutionNotes : String	Action taken
Attribute	origin : InspectionType	POST_RETURN issues count against the club that held the kit; PRE_HANDOVER issues count against nobody
Method	recordIssue(issue)	Stores the description (must not be blank). If an OPEN issue for the item already exists, adds a note to it instead
Method	resolveIssue(notes, itemConditionAfterResolution)	Records the action taken (notes must not be blank). Sets the item to GOOD when it is repaired, or records a replacement that is GOOD. Status stays OPEN
Method	closeCase()	Refused if notes are missing or the item is not GOOD. Otherwise sets RESOLVED. Refused if already RESOLVED

4.8 ReturnHistory

Owned by a club. Turns past returns into a score that ranks the club when requests clash.

Kind	Member	Description
Attribute	clubId : String	Club this history belongs to
Attribute	completedCount : Integer	Reservations of the club that reached COMPLETED
Attribute	expiredCount : Integer (added)	Reservations of the club that reached EXPIRED (no-shows)
Attribute	goodReturns : Integer	Completed reservations returned on time with every item GOOD
Attribute	lateReturns : Integer	Completed reservations returned after the booked end time (information only)
Attribute	openReturnIssues : Integer	POST_RETURN issues of this club that are not yet closed
Attribute	lostWindowsInRow : Integer	Consecutive windows lost with LOST_PRIORITY; reset to 0 when the club wins. CONFLICT does not change it
Method	recordOutcome(onTime, allItemsGood)	Called once per completed return
Method	recordExpiry()	Called once per expired reservation; increments expiredCount
Method	reliabilityScore() -> Number	(goodReturns + 1) / (completedCount + expiredCount + 2). A new club scores 0.50
Method	effectiveScore() -> Number	reliabilityScore + fairness credit, where credit = 0.05 per lost window in a row, at most 0.10

4.9 BookingRequest

A request waiting for its decision window. It is not a Reservation and is never stored as one.

Kind	Member	Description
Attribute	requestId : String	Unique ID of the request
Attribute	clubId, kitId, startTime, endTime	Same meaning as on Reservation
Attribute	receivedAt : DateTime, sequenceNo : Integer	When the request arrived; sequenceNo breaks ties exactly
Attribute	state : RequestState	QUEUED, CONFIRMED or REJECTED
Attribute	rejectReason : RejectReason	Set only when REJECTED
Attribute	reservation : Reservation	Empty until the request is confirmed

Kind	Member	Description
Method	priorityKey() -> Tuple	(has open return issue, minus effective score, sequenceNo, clubId); smaller sorts first
Method	result(timeoutSeconds) -> Reservation	Waits up to the timeout. Returns the Reservation, raises the rejection error, or raises RequestPendingError if still queued. Calling again never creates a new request

4.10 EventKitManager

Coordinates requests, reservations, availability, handover, return, inspection and issue tracking.

Kind	Member	Description
Attribute	kits, reservations, handoverRecords	Collections of kits, reservations and handover records
Attribute	clubs, inspectionReports, disputes	Registered clubs, plus reports and issues kept per reservation
Attribute	requestQueue	Queued requests per kit
Attribute	decisionWindowSeconds : Integer	Window length; 5 by default, 0 for immediate decisions
Method	checkAvailability(startTime, endTime, optional kitId) -> List of EventKit	Kits free for the whole period and its buffers; an empty list means none
Method	makeReservation(clubId, kitId, startTime, endTime) -> BookingRequest	Validates at once (bad input is rejected and never queued), rejects at once if the period clashes with a confirmed booking, otherwise queues the request. With window 0 it decides immediately
Method	resolveWindow(kitId)	Runs when a window closes. Takes the lock, ranks queued requests, confirms in rank order those that fit, rejects the rest, releases the lock, then completes the results. Idempotent
Method	cancelReservation(reservationId, clubId, actorType)	Cancels a CONFIRMED reservation before handover. Raises UnauthorizedCancellationError for any other actor
Method	expireNoShows(now)	Expires every CONFIRMED reservation past its no-show deadline with no handover; updates each club's history
Method	recordHandover(reservationId, givenAt)	Creates the handover record for a CONFIRMED reservation after canBeHandedOver passes. Marks the kit OUT
Method	verifyKitBeforeHandover(reservationId, results, notes, verifiedAt) -> InspectionReport	Verifies every item. A failed check opens or updates issues and blocks handover until a new check passes
Method	recordReturn(reservationId, observed conditions, notes, returnedAt) -> InspectionReport	Records the physical return, completes the reservation, marks the kit IN_INSPECTION, inspects items, opens or updates issues, and updates the club's ReturnHistory. Late returns are counted
Method	releaseUnusedPeriod(reservationId, returnedAt)	Applies early-return release when the kit setting allows it and the inspection passed
Method	handleDispute(reservationId, issue) -> DisputeResolution	Records a missing or damaged item issue, or adds a note to the OPEN issue for that item
Method	resolveDispute(disputeId, notes, itemConditionAfterResolution)	Organizer step one. Records notes and the item's condition after repair or replacement
Method	closeDispute(disputeId)	Organizer step two. Closes only when the item is GOOD. Lowers openReturnIssues for POST_RETURN issues
Method	recoverDecisionWindow(kitId)	Re-runs resolveWindow for a window that has passed its deadline without completing. Safe to repeat
Method	invalidateStaleVerification(now)	Clears currentVerification on every kit whose verification is older than verificationValidityMinutes
Method	addKit, registerClub	Setup: put a kit or a club into the system. Only organizers register clubs

5. UML Class Diagram

The classes connect through Reservation: the club, the kit and every handover, inspection and issue record link to it. A BookingRequest becomes a Reservation only when it wins its window. Green classes are new in this revision.

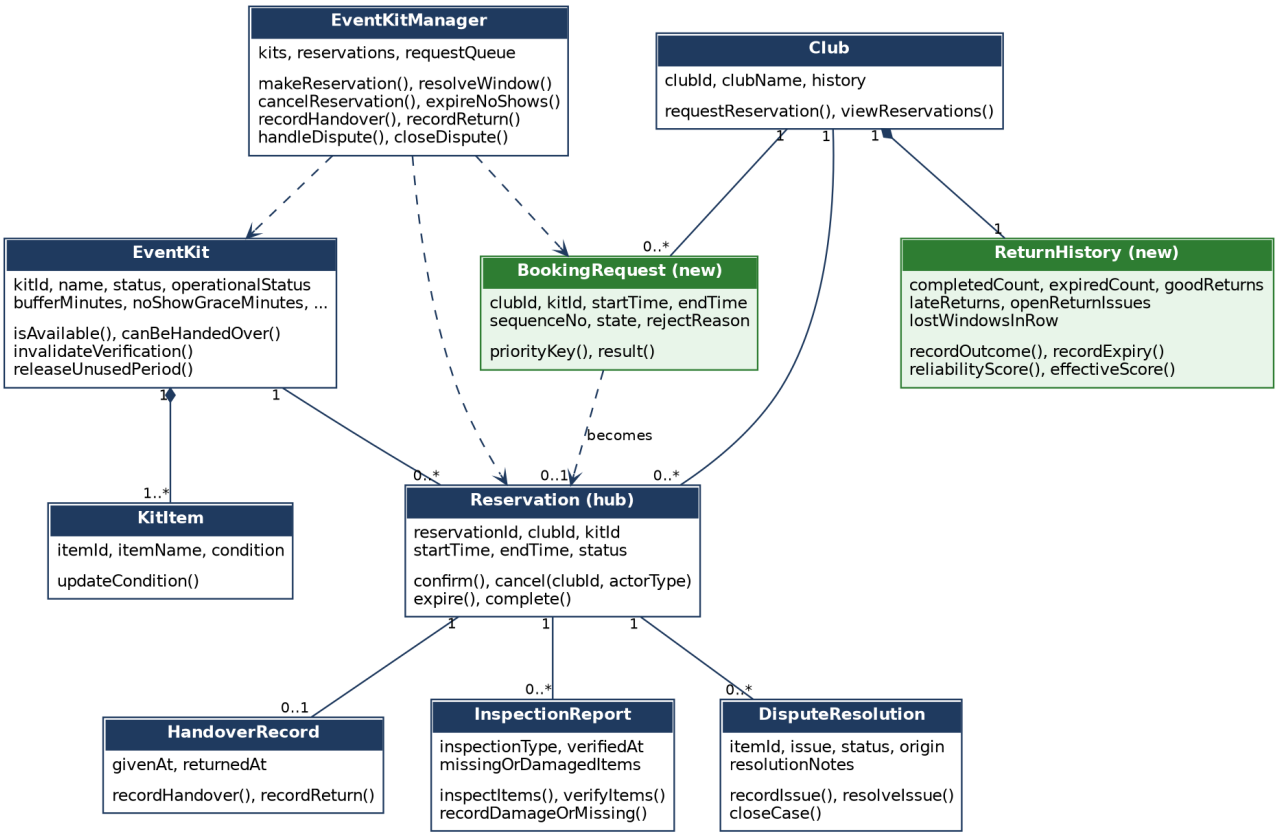


Figure 1. UML class diagram: 10 classes, multiplicities on each link, dashed arrows show the manager coordinating. Full attributes and methods are in section 4.

6. Relationships Between Classes

Relationship	Multiplicity	Explanation
Club - Reservation	1 to 0..*	A club can make many reservations over time.
EventKit - Reservation	1 to 0..*	A kit can be reserved many times, but never for overlapping periods, and always with the buffer between bookings.
EventKit - KitItem	1 to 1..*	A kit contains one or more items; the items belong to the kit (composition).
Reservation - HandoverRecord	1 to 0..1	A reservation has at most one record holding handover and return times.
Reservation - InspectionReport	1 to 0..*	A verification before handover, an inspection after return, and a new verification after a failed one.
Reservation - DisputeResolution	1 to 0..*	A reservation can have issues awaiting resolution. Each issue is for one item.
Club - ReturnHistory	1 to 1	Every club has exactly one history (composition).
Club - BookingRequest	1 to 0..*	A club can send many requests over time; only one QUEUED request per kit per window (rule 9).
BookingRequest - Reservation	1 to 0..1	A request becomes a reservation only when it is confirmed.
EventKitManager - all records	1 to 0..*	The manager coordinates kits, requests, reservations, handovers, inspections and disputes.

7. Business Rules

Fifteen rules govern the system. Section 14 shows where each is enforced and which test scenario proves it.

#	Rule	What it means in this design
1	Availability	A kit cannot be reserved by two clubs for overlapping periods, and two bookings must keep at least the 15-minute buffer. The check covers the whole requested period.
2	Reservation details	A reservation needs a valid club, valid kit, start before end, start not in the past, and duration within the configured minimum and maximum.
3	Handover	The system records when the kit is given to the club, only inside the handover window and only when the kit is physically ready.
4	Return	The system records the actual physical return and checks every kit item. A late return is recorded as late.
5	Inspection	Missing, damaged or not-working items must be recorded in an inspection report.
6	Dispute handling	An issue stays open until an organizer records resolution notes and the item is GOOD again. Closing is refused otherwise.
7	Concurrent requests	Simultaneous requests are decided under the manager lock, so only valid winners are confirmed.
8	Verification before handover	Every item must be verified as GOOD, and the verification must still be valid at handover.
9	Priority on simultaneous requests	Requests in the same decision window are ranked by return history. A lower-ranked request is confirmed only if it still fits.
10	Kit back before next handover	A kit cannot be handed over while an earlier handover of it has not been returned, however late.
11	No-show expiration	A CONFIRMED reservation with no handover becomes EXPIRED at start plus the no-show grace period. It stops blocking future periods and counts as not good.
12	Physical kit state	Reservation status and physical kit state are tracked separately. OUT, IN_INSPECTION and NEEDS_REPAIR block handover.
13	Verification freshness	A verification expires after verificationValidityMinutes and is invalidated by any item-condition change.
14	Cancellation ownership	Only the reservation-owning club or an organizer can cancel, and only before handover.
15	Early return	If enabled, an early clean return may release the unused period; otherwise the original period stays blocked.

8. State Transitions

Reservations, handover records, issues and requests only move forward. The kit's summary status and operational status move back and forth.

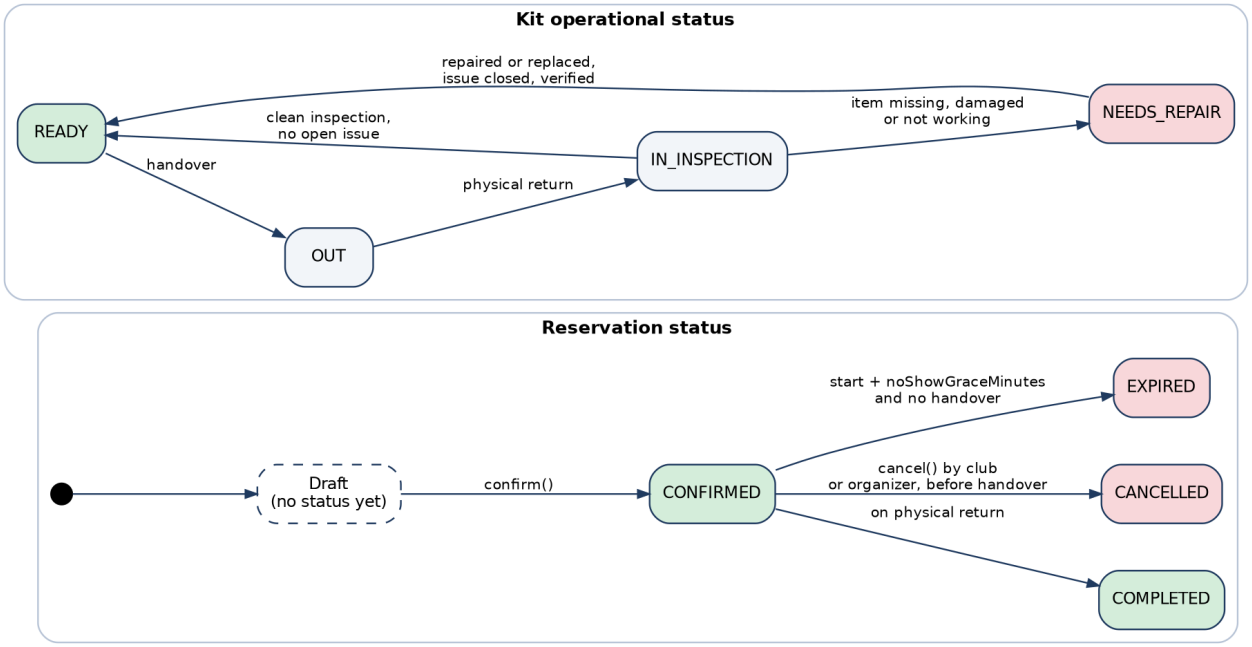


Figure 2. Reservation status (left) and kit operational status (right). KitStatus AVAILABLE/RESERVED is only a summary of CONFIRMED reservations.

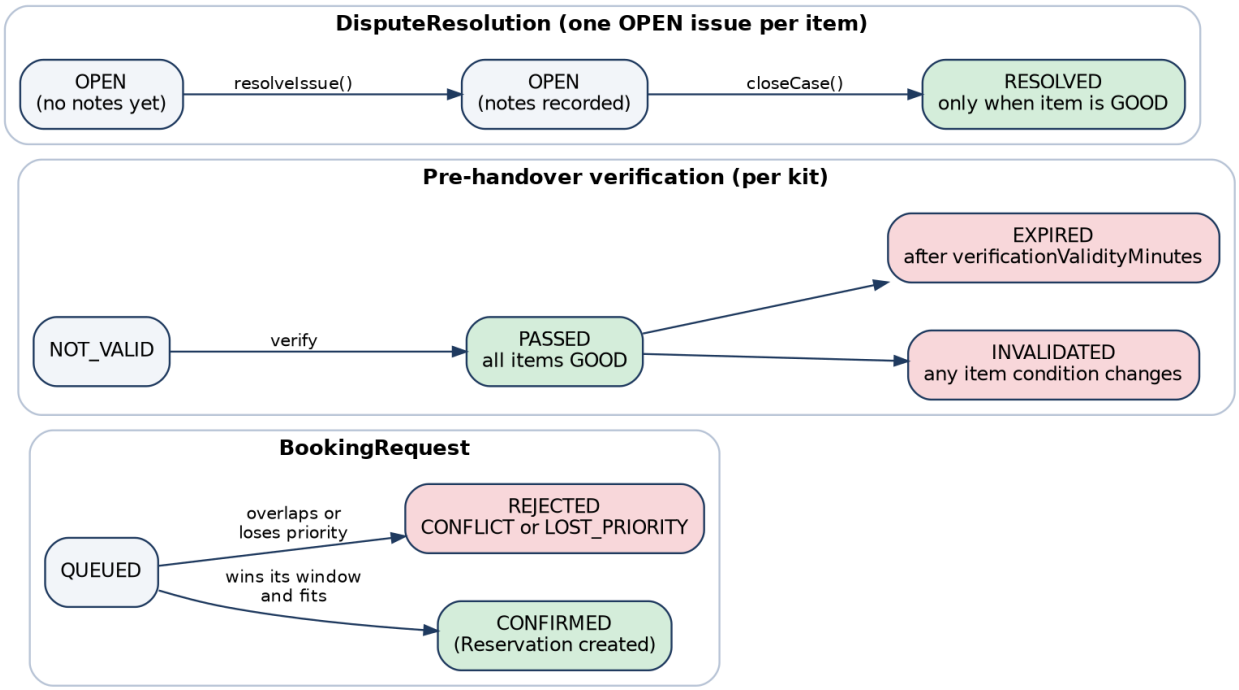


Figure 3. BookingRequest, pre-handover verification and DisputeResolution state transitions.

A reservation can be cancelled only while CONFIRMED and before handover. A handover needs a passing and valid verification, a READY kit and a time inside the handover window. A late return keeps the kit in IN_INSPECTION until the physical return is recorded and inspected.

9. Workflow

A request ends rejected, cancelled, expired or completed. A failed item check blocks the handover. A faulty item at return keeps the kit in NEEDS_REPAIR and the issue open.

Part 1: request to confirmed booking

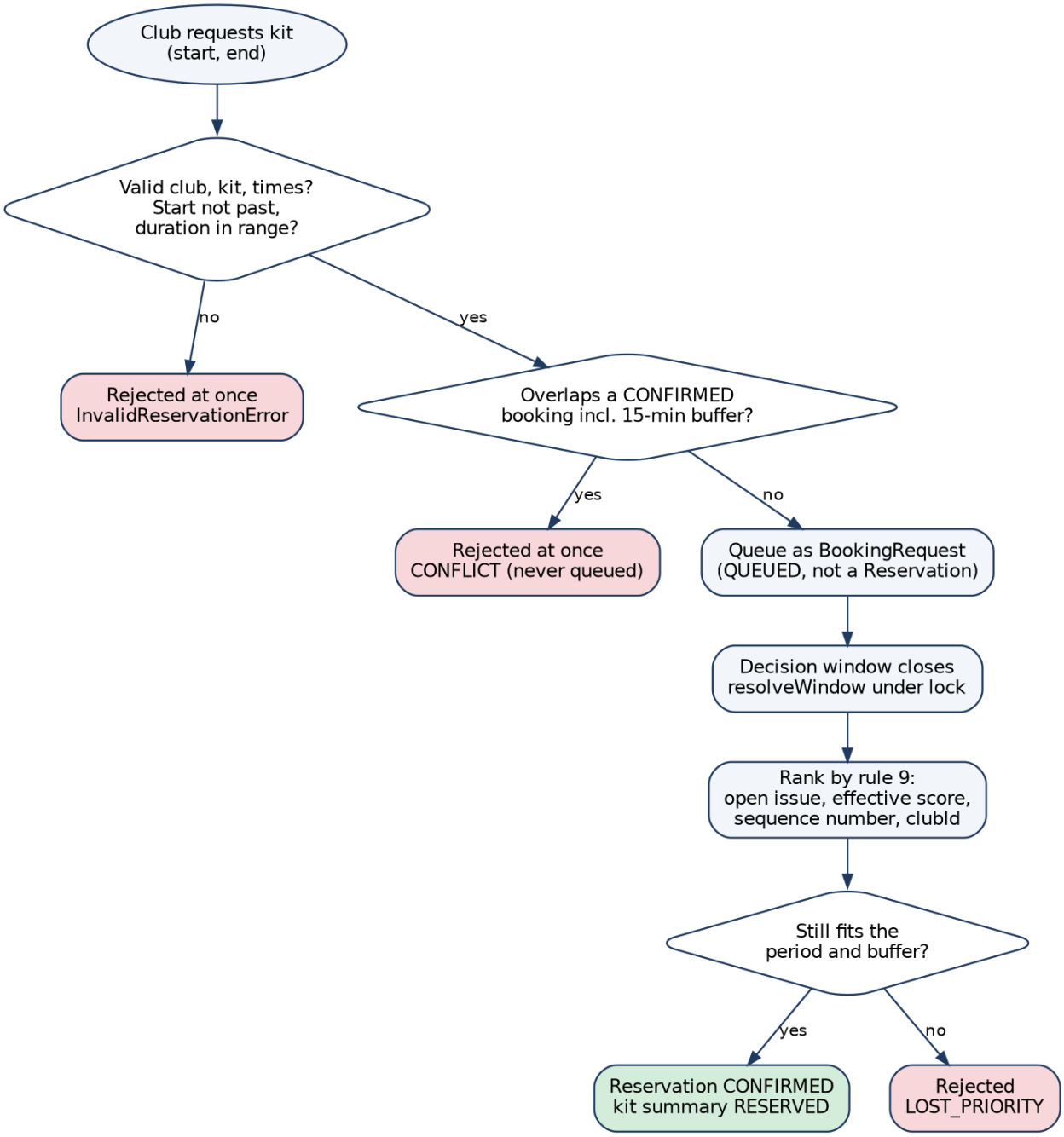


Figure 4. Request to confirmed booking, including the decision window and ranking.

- Validate club, kit, times and duration. Reject a start in the past or a duration out of range at once.
- Reject at once a request that conflicts with an already confirmed booking (CONFLICT).
- Queue the other valid requests for the decision window (or decide at once when the window is 0).
- When the window closes, rank the queued requests by return history (rule 9).
- Confirm winners in one locked step; reject losers with LOST_PRIORITY when a same-window winner blocks them.

Part 2: confirmed booking to handover

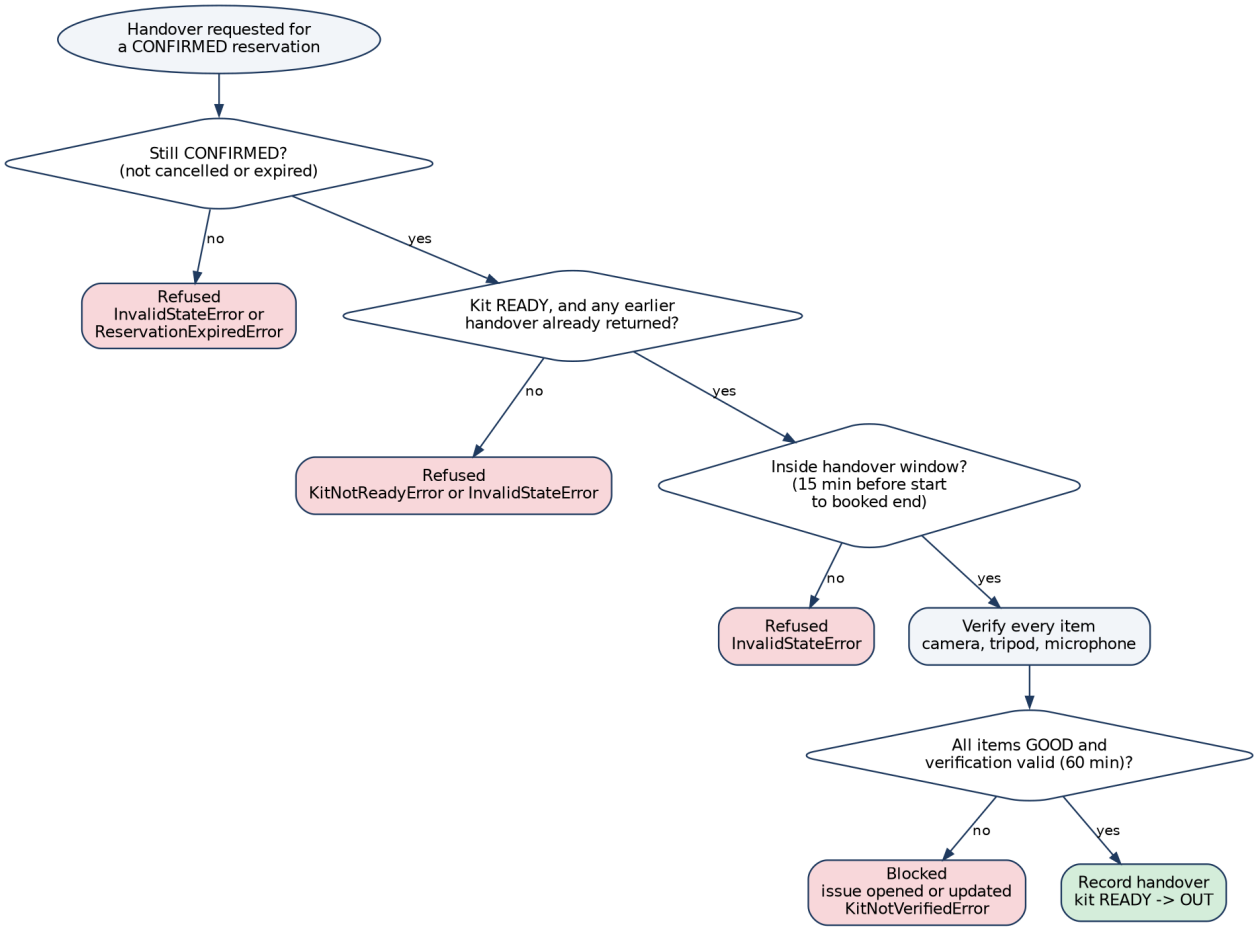


Figure 5. Handover checks, in order. Any refusal leaves the kit and the reservation unchanged.

- Confirm the reservation is still CONFIRMED (not cancelled or expired).
- Check the kit is READY (not OUT, IN_INSPECTION or NEEDS_REPAIR) and that the earlier handover, if any, has been returned.
- Check the time is inside the handover window (from 15 minutes before start to the booked end).
- Verify every item. Handover needs all items GOOD and a valid verification.
- Record the handover and set the kit to OUT.

Part 3: return to closed issue

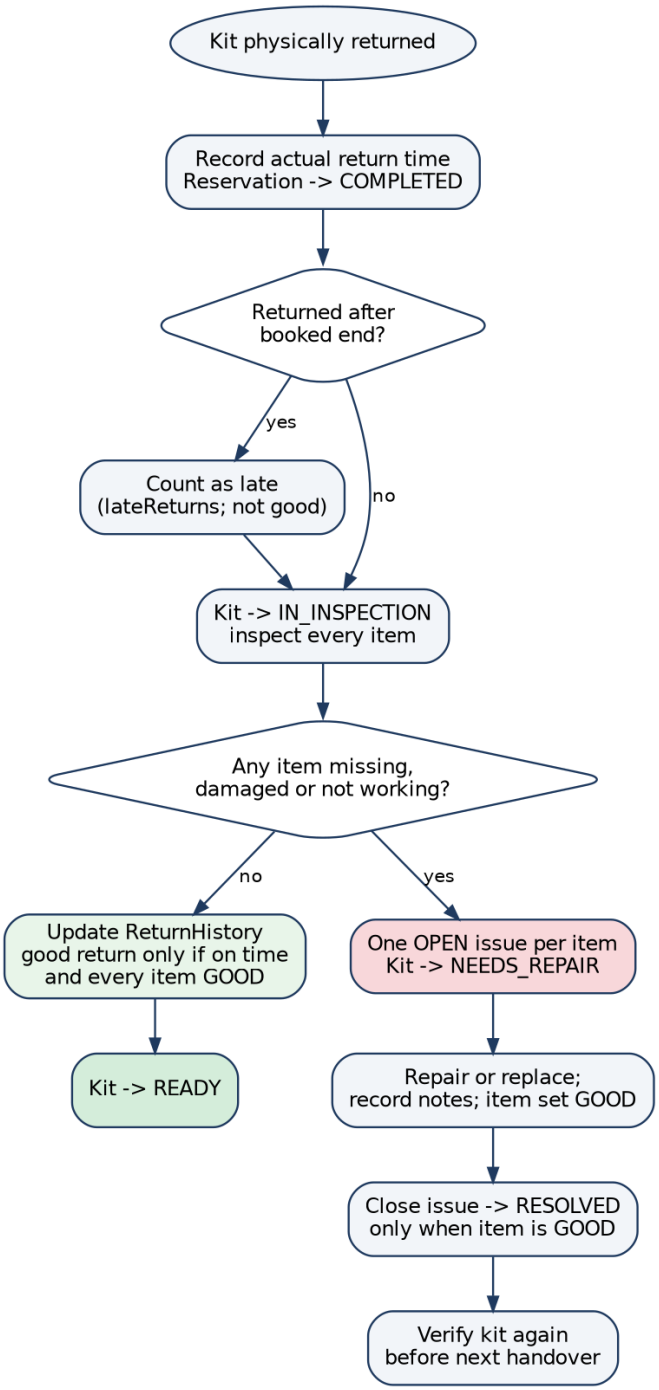


Figure 6. Return, inspection, issue handling and re-verification.

- Record the actual physical return time. Mark the reservation COMPLETED (or record it as late).
- Set the kit to IN_INSPECTION and inspect every item.
- If no item is missing, damaged or not working, set the kit to READY and update the history.
- Otherwise open one OPEN issue per item, set the kit to NEEDS_REPAIR and update the history.
- Repair or replace the item and record resolution notes that set it to GOOD.
- Close the issue. Verify the kit again before the next handover.

10. Validation and Error Handling

Every invalid request is rejected with a specific error. A rejected request changes nothing: no reservation is stored, no item condition is touched. The table below is the complete list for Revision 4; it replaces the Revision 2 and

Revision 3 tables and keeps every error from both.

Error	Raised when
UnknownClubError	The club is not registered
UnknownKitError	The kit is not registered
UnknownReservationError	The reservation ID does not exist
UnknownItemError	An inspection or verification names an item that is not in the kit
UnknownDisputeError	The issue ID does not exist
InvalidReservationError	Missing or non-date times; start not before end; start in the past; duration below minimum or above maximum
ReservationConflictError	The period overlaps or comes within the buffer of a non-cancelled reservation. Carries CONFLICT or LOST_PRIORITY
DuplicateRequestError	The club already has a QUEUED request for this kit in the open window
RequestPendingError	A caller's wait for a request result ends before the window closes. Retryable; a retry returns the same result and creates no request
InvalidStateError	The action is not allowed now: cancel after handover; return before handover; second handover or return; handover before the handover window; handover while an earlier handover is outstanding; completing a reservation that is not CONFIRMED; closing an issue already closed
UnauthorizedCancellationError	The caller is neither the reservation-owning club nor an organizer
ReservationExpiredError	An operation tries to use a reservation after its no-show deadline
KitNotReadyError	The kit is OUT, IN_INSPECTION or NEEDS_REPAIR
KitNotVerifiedError	No current valid pre-handover verification exists for the reservation
VerificationExpiredError	The previous verification is older than verificationValidityMinutes
IncompleteVerificationError	A pre-handover verification does not give a result for every item
UnresolvedIssueError	An issue is being closed while its item is not GOOD
DisputeError	An issue description or resolution note is blank, or a close is tried without resolution notes
InvalidKitError	A kit is defined with no items

11. Example Scenarios

Example A: Club A gets the kit, Club B is turned away, and a damaged microphone is tracked to closure (window 0)

- 1. Club A requests 10:00 AM to 11:00 AM. No overlap, so it is confirmed. The reservation is CONFIRMED and the kit shows RESERVED.
- 2. Club B requests 10:30 AM to 11:30 AM. It overlaps Club A from 10:30 to 11:00, so it is rejected with CONFLICT and nothing is created for Club B.
- 3. Club B tries again from 11:10 AM. Club A's booking ends at 11:00, so the gap is 10 minutes and the request is rejected. A request from 11:15 AM is accepted.
- 4. Before handover, the organizer verifies the camera, tripod and microphone. All work, so the handover is recorded and the kit shows OUT.
- 5. Club A returns the kit at 11:00 AM. The reservation is marked COMPLETED and the kit goes to IN_INSPECTION.
- 6. The organizer inspects the items: camera and tripod are GOOD, the microphone is DAMAGED. One OPEN issue is opened for the microphone. The kit goes to NEEDS_REPAIR. Club A's return is counted as not good.
- 7. The organizer records the action taken. The issue is still OPEN, because the microphone is not yet GOOD.
- 8. The microphone is repaired and the organizer records notes that set it to GOOD. The issue is closed and RESOLVED. The kit goes to READY.

- 9. Before the next handover, the organizer verifies all items again. If the microphone still fails, the existing OPEN issue for it gets a new note and the handover stays blocked.

Example B: two clubs ask at the same time (window 5 seconds)

Photography has 4 completed returns, all good (score 0.83). Music has 3 completed returns, 1 good (score 0.40). Both ask for 2:00 PM to 3:00 PM within one window, and Music's request arrives first.

- 1. Both requests are valid and no confirmed booking blocks them, so both are queued as BookingRequests. Neither is a Reservation yet.
- 2. The window closes. The ranking is Photography (0.83), then Music (0.40). Music's earlier arrival does not matter, because arrival order only breaks ties.
- 3. Photography is confirmed and the kit shows RESERVED. Music is rejected with LOST_PRIORITY, and its lostWindowsInRow becomes 1.
- 4. Music asks again for 3:15 PM to 4:15 PM. The gap after Photography's booking is exactly 15 minutes, so it is confirmed.
- 5. Photography returns on time with every item GOOD: 5 completed, 5 good, new score 6/7 = 0.86.

12. Ranking Worked Example

Club	Completed	Good returns	Expired	Score	Open return issue	Rank
Photography	4	4	0	(4+1)/(4+0+2) = 0.83	no	1
Robotics (new)	0	0	0	(0+1)/(0+0+2) = 0.50	no	2
Music	3	1	0	(1+1)/(3+0+2) = 0.40	no	3
Drama	5	5	0	(5+1)/(5+0+2) = 0.86	yes	4 (rule 6 puts it last)

Rule 9 ordering (priorityKey)

- 1. Clubs with no open return issue come before clubs that still have one.
- 2. Higher effective score first: reliability score plus the fairness credit.
- 3. Earlier sequence number first. Arrival order only breaks ties.
- 4. Smaller clubId first, so the result is the same on every run.

13. Concurrency, Priority and Recovery

13.1 Why a decision window

A lock alone only gives first come, first served. To prefer the better club, the manager must see competing requests together. Requests for one kit are collected for a short window (default 5 seconds). The window for a kit opens when a valid request arrives and none is open, and closes after decisionWindowSeconds. Requests for different kits have independent windows.

13.2 What happens when a window closes (resolveWindow)

- 1. Take the manager's lock.
- 2. Collect the QUEUED requests of the kit and sort them by priorityKey.
- 3. In order, check kit.isAvailable(period). If true, create the reservation, confirm it, reset the club's lostWindowsInRow and mark the request CONFIRMED.
- 4. Otherwise mark the request REJECTED: LOST_PRIORITY when the blocking booking was confirmed in this same window, CONFLICT when it was confirmed earlier. Add 1 to lostWindowsInRow only for LOST_PRIORITY. A CONFLICT rejection leaves lostWindowsInRow unchanged.
- 5. A rejected request does not block the ones after it; each is checked against what is actually confirmed.

- 6. Update the kit summary status, release the lock, then complete the result of every request.

The result is greedy by priority: it gives the kit to the best-ranked club, not necessarily the largest number of bookings. This is intended, because the goal is to reward good returns.

13.3 Locking rules

- 1. The window timer never holds the lock while it waits. Only resolveWindow takes the lock.
- 2. makeReservation (validation and immediate rejection), resolveWindow, cancel, handover, return, expiry and issue changes all use the same lock, so a reservation is never seen half-updated.
- 3. The lock is re-entrant, because recording a return can open an issue from inside the same locked call.
- 4. resolveWindow is idempotent. A second call finds no QUEUED requests and does nothing.
- 5. With window 0 each request is resolved at once under the lock.

13.4 Recovery and bounded waits

- A caller waits for its result for a bounded time. If the window has not completed, the caller gets RequestPendingError and may retry.
- Retrying the result lookup never creates another request; it returns the same final result.
- If a window's deadline has passed without completion, recoverDecisionWindow runs resolveWindow again. Queued requests are in memory; a restart loses them and callers must resubmit.

13.5 Physical-state safety

A reservation may be CONFIRMED while the kit is physically READY. Handover changes the operational state to OUT. Return changes it to IN_INSPECTION. Inspection then sends it to READY or NEEDS_REPAIR. A late return never allows the next handover before the kit is physically back, inspected and verified.

14. Rule Traceability

Rule	Enforced in	Proved by scenario
1. Availability	EventKit.isAvailable, EventKitManager.makeReservation, EventKitManager.resolveWindow	2, 3, 4, 15, 16, 25, 26, 29, 31
2. Reservation details	EventKitManager.makeReservation, Reservation.confirm	1, 5, 39, 40
3. Handover	EventKitManager.recordHandover, HandoverRecord.recordHandover, EventKit.canBeHandedOver	6, 8, 10, 17, 20, 41
4. Return	EventKitManager.recordReturn, HandoverRecord.recordReturn, ReturnHistory.recordOutcome	6, 8, 9, 27, 34
5. Inspection	InspectionReport.inspectItems, InspectionReport.verifyItems, InspectionReport.recordDamageOrMissing	7, 18
6. Dispute handling	DisputeResolution.resolveIssue, DisputeResolution.closeCase	7, 11, 12, 35
7. Concurrent requests	The manager lock around makeReservation and resolveWindow	13, 14, 29
8. Verification before handover	EventKitManager.verifyKitBeforeHandover, EventKit.canBeHandedOver, EventKit.invalidateVerification	17, 18, 19, 20, 36, 37
9. Priority on simultaneous requests	EventKitManager.resolveWindow, BookingRequest.priorityKey, ReturnHistory.reliabilityScore, ReturnHistory.effectiveScore	21, 22, 23, 24, 25, 26, 27, 30, 31
10. Kit back before next handover	EventKit.canBeHandedOver, EventKitManager.recordHandover	28, 34
11. No-show expiration	Reservation.expire, EventKitManager.expireNoShows, ReturnHistory.recordExpiry	32
12. Physical kit state	EventKit.operationalStatus transitions, EventKit.canBeHandedOver	33, 34

Rule	Enforced in	Proved by scenario
13. Verification freshness	EventKit.invalidateVerification, EventKitManager.invalidateStaleVerification	36, 37
14. Cancellation ownership	Reservation.cancel, EventKitManager.cancelReservation	10, 38
15. Early return	EventKitManager.releaseUnusedPeriod, EventKit.releaseUnusedPeriod	42, 43

15. Test Scenarios

Forty-four scenarios cover every rule. Scenarios 1 to 20 run with window 0. Scenarios 1 to 31 keep the Revision 2 expected results. Scenarios 32 to 44 are the Revision 3 additions, with 35 and 36 updated for the Revision 4 decisions.

#	Scenario	Expected result	Rule
1	A club requests a free period	CONFIRMED; kit shows RESERVED; reservation appears in the club's list	1, 2
2	A second club requests an overlapping period	Rejected with CONFLICT; nothing created for the second club	1
3	A second club requests a period starting exactly when the first ends	Rejected; no 15-minute gap kept	1
4	A confirmed reservation is cancelled, then an overlapping period is requested	Cancelled slot is free; kit shows AVAILABLE if no other CONFIRMED reservation remains	1, 14
5	Request with unknown club, unknown kit, start after end, start equal to end, or a missing or non-date time	Specific error; nothing stored	2
6	Verify all items, hand over, then return with all items GOOD	Both times recorded; reservation COMPLETED; no issue opened; kit goes OUT then READY	3, 4
7	Return with the microphone DAMAGED	Inspection report lists the microphone; one OPEN issue; kit NEEDS_REPAIR	4, 5, 6
8	Return before handover, or a second return	Refused with InvalidStateError	3, 4
9	Return naming an item not in the kit	Refused with UnknownItemError; reservation and handover record unchanged	4
10	Cancel after the kit was handed over, or cancel by another club	Refused	3, 14
11	Close an issue with no notes; resolve with blank notes	Refused with DisputeError; issue stays OPEN	6
12	Record notes, set the item GOOD, close, then close again	After notes: still OPEN. After close: RESOLVED. Second close refused	6
13	Two clubs request overlapping periods at the same moment, repeated many times	Exactly one confirmed, one rejected every time	7
14	Ten clubs request overlapping periods at the same moment	Exactly one confirmed, nine rejected	7
15	A second club requests a period starting exactly 15 minutes after the first ends	Confirmed	1
16	A second club requests a period starting 14 minutes after the first ends	Rejected; gap too short	1
17	Hand over without a verification	Refused with KitNotVerifiedError; no handover record	8
18	Verification where the microphone does not work	Report lists the microphone; one OPEN issue; handover stays blocked	5, 8
19	Verification that gives no result for one item	Refused with IncompleteVerificationError; nothing stored	8
20	Fix the item, verify again with all items GOOD, then hand over	Handover allowed	8

#	Scenario	Expected result	Rule
21	Club with good history and club with poor history request overlapping periods in the same window; the poor one arrived first	Good-history club confirmed; other rejected with LOST_PRIORITY; nothing created for it	9
22	Two clubs with equal history request overlapping periods in the same window	The request received first is confirmed; the other is rejected	9
23	A new club (score 0.50) against a club with 0 good returns out of 3 (score 0.20)	The new club is confirmed	9
24	Club with a high score but an OPEN issue from its own return, against a clean club with a lower score	Clean club confirmed. After the issue is closed, ranking is by score again	9
25	Two requests in the same window for periods that do not clash (buffer kept)	Both confirmed, whatever their priority	1, 9
26	A (best) overlaps B (middle); C (lowest) overlaps B only	A and C confirmed; B rejected; B's rejection does not block C	1, 9
27	Return after the booked end time with every item GOOD	Reservation COMPLETED; counted as late and not good in the history	4, 9
28	Hand over a kit whose previous handover has not been returned	Refused with InvalidStateError; no handover record	10
29	Decision window set to 0	Each request decided at once in arrival order; scenarios 1 to 20 behave as before	1, 7
30	Same club sends a second request for the same kit in the open window	Refused with DuplicateRequestError; first request unchanged	9
31	Request overlaps a booking confirmed before the window opened, from a club with better history	Rejected at once with CONFLICT; a confirmed booking is never taken away	1, 9
32	Confirmed reservation never handed over	After the no-show deadline the reservation is EXPIRED; its period is released; club's history counts it as not good	11
33	Kit has a future reservation but is physically in storage	Reservation may be CONFIRMED while operationalStatus stays READY; physical state is not inferred from a future booking	12
34	Late return while next booking exists	Next handover refused with InvalidStateError until the kit is physically returned, inspected and verified	4, 10, 12
35	Issue closed while item is still damaged	Close refused with UnresolvedIssueError. Closing is allowed only after the item is set to GOOD by repair or a GOOD replacement	6
36	Verification older than validity period	Handover refused with VerificationExpiredError; a new verification is required	8, 13
37	Item condition changes after successful verification	Verification invalidated; handover refused with KitNotVerifiedError until a new passing verification	8, 13
38	Another club tries to cancel a reservation	Refused with UnauthorizedCancellationError; reservation unchanged	14
39	Reservation starts in the past	Rejected with InvalidReservationError; no reservation or request created	2
40	Reservation exceeds maximum duration	Rejected with InvalidReservationError	2
41	Handover before the handover lead time	Rejected with InvalidStateError	3
42	Early clean return with release setting enabled	Blocked period ends at return plus buffer; a new compatible reservation may use the released time	15
43	Early return with release setting disabled	Original booking period continues to block overlapping requests	15
44	Decision window caller times out and retries	RequestPendingError on timeout; retry returns the same final result; no duplicate reservation	7, 9

16. Conflict Resolution Log

This log records every conflict found in Revision 2 (Appendix A) and in the Revision 3 review, with the decision now in force.

#	Conflict	Revision 4 decision
1	No-show: a CONFIRMED reservation is never handed over	EXPIRED status at start + noShowGraceMinutes (30). Counts as not good in history (reliability denominator).
2	Late return eats the next gap	A late return is counted in lateReturns and is not good. Next handover waits for the physical return, inspection and verification. No penalty.
3	Kit status flag is misleading	KitStatus is a summary only. KitOperationalStatus holds physical state. Availability uses the overlap check only.
4	Issue closed but item still damaged; one issue per failed verification	One OPEN issue per item; repeats add notes. Closing requires the item GOOD (repair, or a GOOD replacement).
5	Verification goes stale	verificationValidityMinutes (60) and invalidation on any item-condition change; owned by EventKit.
6	Cancellation has no owner check	cancel takes clubId and actorType; only the owning club or an organizer may cancel.
7	Gaming the ranking	Only organizers register clubs. Active-booking limits remain an operational policy; implementations may enforce them.
8	Past, very short or very long bookings	Start in the past rejected; duration between 30 minutes and 3 days.
9	Early return still blocks the period	releaseUnusedTimeAfterEarlyReturn (false by default); applies only after a passing inspection.
10	Handover outside the booked period	Handover allowed from 15 minutes before start until the booked end.
11	Window timer failure	Bounded caller wait with RequestPendingError, retryable lookup, and idempotent resolveWindow with recoverDecisionWindow.
R3-a	Revision 3 change log says an issue can close without a condition change; Appendix A and scenario 35 require one	Resolved: closing requires the item GOOD again. No other exception.
R3-b	Revision 3 workflow opened a new issue on every failure	Resolved: one OPEN issue per item, as in Appendix A item 4.
R3-c	Late return could be read as a penalty	Resolved: counted in history only, as in Revision 2.
R3-d	Verification owner split between InspectionReport and EventKit	Resolved: EventKit owns validity; InspectionReport stores data.
R3-e	IN_INSPECTION had no path to READY when inspection passed	Resolved: IN_INSPECTION -> READY on a clean inspection.
R3-f	Revision 3 error table replaced Revision 2's	Resolved: one merged table in section 10.
R3-g	New settings (noShowGraceMinutes, minimumBookingMinutes, maximumBookingDays) had no owner	Resolved: all kit settings live on EventKit; decisionWindowSeconds on EventKitManager.
R3-h	A CONFLICT rejection's effect on lostWindowsInRow was not stated	Resolved: CONFLICT does not change lostWindowsInRow.

Recommended defaults

Setting	Default
bufferMinutes	15
decisionWindowSeconds	5
noShowGraceMinutes	30
verificationValidityMinutes	60
minimumBookingMinutes	30

Setting	Default
maximumBookingDays	3
handoverLeadMinutes	15
releaseUnusedTimeAfterEarlyReturn	false

Revision 4 is the implementation-ready specification. The Revision 2 concurrency and priority behaviour is kept. The Revision 3 additions and the review fixes above make every operational case explicit.